Lançamento do Projétil Interativo

Autores:
-Carolina Cordeiro - 119320
-Guilherme Almeida - 120069
-Pedro Sousa - 119688

Turma: PI2

Data de realização: 09/05/2024 – 24/05/2024

1. Resumo

O nosso trabalho visa dinamizar e tornar mais apelativo e lúdico o estudo da mecânica do projétil. Para tal, criámos um simulador de lançamento oblíquo onde o usuário pode definir as condições iniciais de lançamento, tais como a velocidade, ângulo de disparo e até a gravidade aplicada. Para além do simulador também desenvolvemos um jogo cuja finalidade é acertar um alvo a partir dos dados iniciais introduzidos pelo utilizador.

2. Introdução

O tema do projeto é o lançamento de projéteis e tem como finalidade criar um jogo interativo, bem como um simulador que permita ao utilizador explorar a mecânica de um projétil e observar as alterações entre diferentes velocidades e ângulos iniciais, assim como experienciar diferentes acelerações gravíticas. Nesta simulação virtual é desprezada a resistência do ar, assim como outros tipos de atrito e movimento de rotação, para que se torne mais simples e acessível de jogar e aprender as equações do movimento.

Este jogo pode também servir como uma curiosidade para quem se interessa pelo espaço e pelos diferentes astros, assim como qual seria a experiência de atirar um objeto em condições impossíveis de recriar na vida real, como gravidades consideravelmente baixas e altas. Com isto, pretendemos não só que se aprenda a dinâmica de um projétil, mas que o utilizador se divirta e se sinta desafiado a acertar no alvo utilizando conhecimentos de física.

Para a realização deste trabalho é necessário conhecer previamente as equações do movimento de um projétil:

$$v_{y_0} = \sin(\theta) v_0$$
$$v_x = \cos(\theta) v_0$$

$$y = y_0 + v_{y_0} - \frac{gt^2}{2}$$
$$x = x_0 + v_x t$$

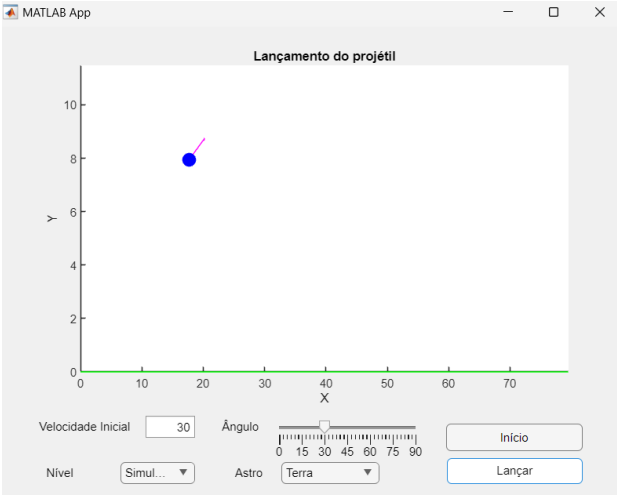
Para desenvolver o nosso trabalho utilizámos a ferramenta *App Designer* do *MATLAB* para criar uma interface interativa onde o utilizador define as condições iniciais e visualiza a animação do lançamento, atualizando os dados iniciais sempre que o programa é executado.

Na execução do trabalho usámos vários algoritmos:

- Expressões *switch* para atribuir a cada astro um valor de aceleração gravítica, bem como associar cada nível ao seu gráfico correspondente
- Ciclos *while* para traçar a animação do lançamento do projétil
- Expressões *if* para verificar se o projétil atingiu ou não o alvo, por meio de operações lógicas

3. Desenvolvimento

3.1. Simulador



Como referido anteriormente, a primeira parte do nosso projeto foi o desenvolvimento de um simulador de projétil. Para tal, o nosso programa recolhia as informações iniciais e através das equações do movimento, calculava e desenhava a posição do projétil em cada instante. Com o intuito de tornar a visualização do voo mais agradável e facilitar o seu estudo, decidimos tornar a janela de animação dinâmica, isto é, dependendo das condições iniciais inseridas, os limites da janela adaptavam-se ao alcance e à altura máxima do projétil. Assim, tivemos de desenvolver as equações da posição para obter tais valores.

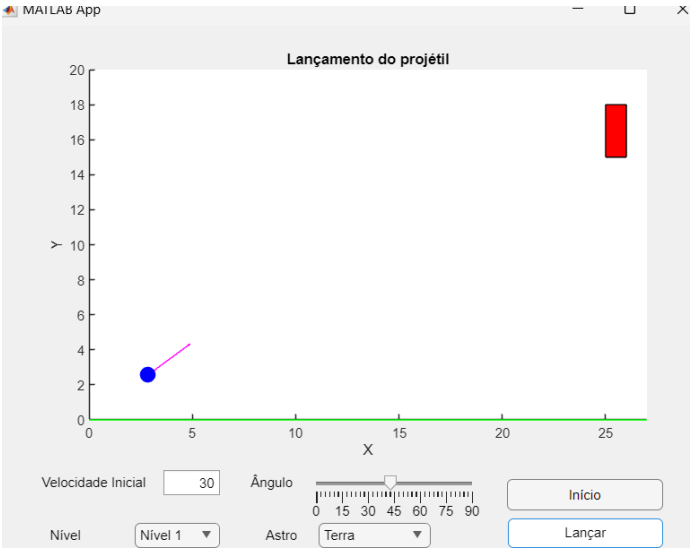
$$t_{voo} = \frac{2v_{y_0}}{g}$$

$$y_{max} = \frac{v_{y_0}^2}{2g}$$

$$x_{max} = v_x t_{voo}$$

Para além do movimento do projétil, optámos por representar também o seu vetor direção para que o utilizador possa mais facilmente observar o comportamento de um projétil durante a sua trajetória. Para criar este vetor, calculámos em cada ponto o módulo da velocidade e as coordenadas do vetor eram as coordenadas do vetor velocidade divididas pelo módulo.

3.2. Nível 1

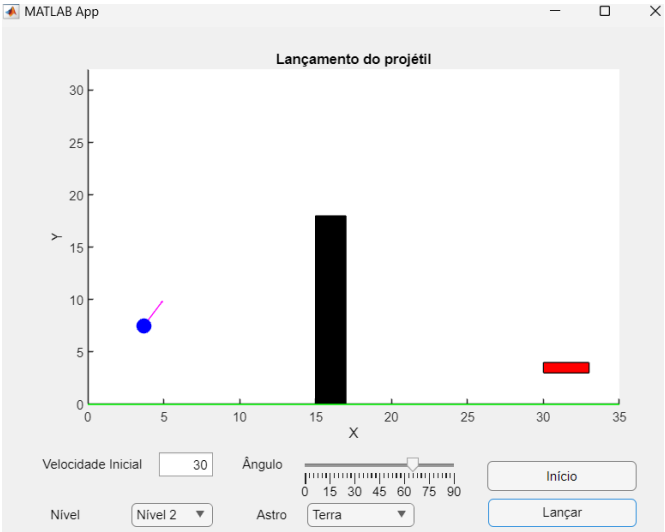


Após ter concluído o simulador, passámos para a produção do jogo, onde o objetivo é acertar num alvo através da escolha das condições iniciais de lançamento. Tratando-se este do nível 1, é naturalmente o mais simples, onde há apenas o alvo a acertar numa posição fixa e existem inúmeras combinações de velocidade inicial e ângulo para ganhar.

Além do código já anteriormente desenvolvido para o simulador, tivemos de adicionar as coordenadas do alvo e desenhá-lo no gráfico. Para o programa saber se o utilizador ganhou ou não, utilizámos uma expressão *if* para verificar se quando o projétil passa pelas coordenadas x do alvo, a sua altura, y, está dentro do intervalo dos y do alvo, se tal se verificar, o programa faz um *pop-up* de uma tela de vitória, caso contrário, a tela é de derrota.

Após testarmos o nosso programa pelas primeiras vezes, apercebemo-nos que no caso do jogador definir uma velocidade muito elevada, mesmo com o projétil a sair dos limites do gráfico, o programa só parava de correr quando este tocasse no chão. Para melhorar a experiência, definimos que caso a coordenada x do projétil ultrapasse o limite referido, o instante de tempo atual igualava-se ao instante de tempo final, desta forma, acabando o ciclo *while* e consequentemente terminando o programa. Foi também desta forma que impedimos o projétil de continuar o seu percurso mesmo após ter acertado o alvo.

3.3. Nível 2

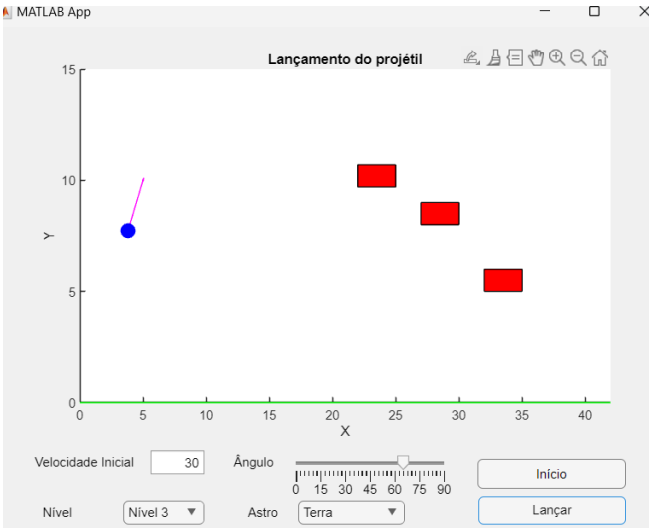


Sendo o programa um jogo, concluímos que este deveria ter vários níveis de modo a possibilitar um retorno à aplicação para fins de lazer e desafio, visto que um jogo simples com apenas 1 nível não proporciona estas condições.

Assim sendo, optamos por gradualmente aumentar a dificuldade com o passar dos níveis com o intuito do utilizador não se sentir frustrado com um aumento de dificuldade grande que lhe dê a ideia de que a dificuldade é injusta. Apesar de que uma opção de “trocar o astro” em que é realizado o lançamento do projétil é disponibilizada, a escolha deste irá corresponder à alteração da constante de aceleração gravítica correspondente a esse astro.

Tendo isto em conta, no nível 2 limitamo-nos a adicionar uma “parede” no meio da janela entre o local de lançamento e o alvo, o qual foi alterado para se encontrar na horizontal, visto que o projétil tem de obrigatoriamente descrever uma trajetória parabólica, ao contrário do nível 1, em que a trajetória do projétil é irrelevante, desde que acerte no alvo. Esta parede é apenas um retângulo preto que é exibido numas coordenadas específicas do gráfico, e no código do nosso programa se (usando a expressão if) o projétil tocar na área cujos vértices correspondem a essas coordenadas, a tela de derrota é exibida.

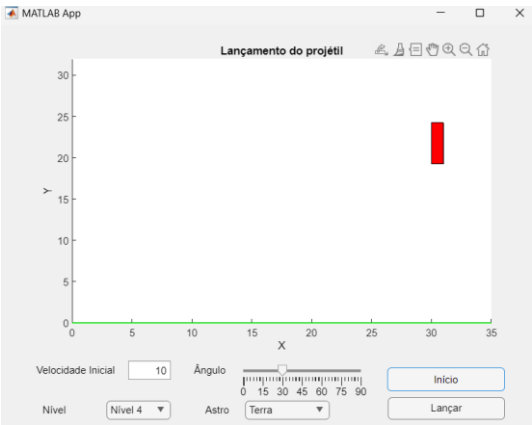
3.4. Nível 3

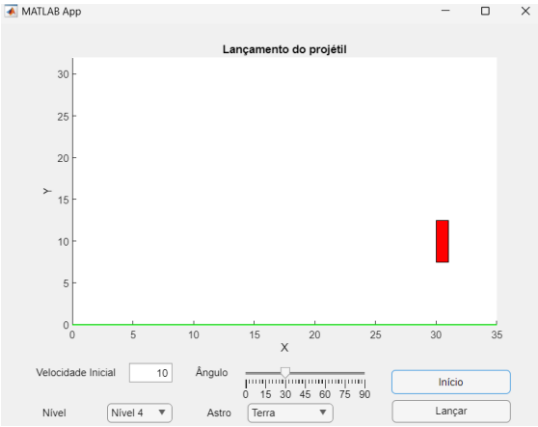


No nível 3 queríamos que o projétil tivesse de acertar em 3 alvos, de forma a obrigar o jogador a selecionar umas condições iniciais especificas que fizesse o projétil passar pelos 3 alvos. Para isso fizemos com que os dois primeiros alvos fossem “checkpoints” obrigatórios e o último seria o alvo real onde o projétil acabaria.

De modo a obter este resultado, mantivemos o código de vitória para o último alvo, mas adicionamos duas condições; criamos uma variável S1 correspondente ao 1º alvo e outra S2 correspondente ao 2º alvo as quais começavam com o valor 0, quando o projétil atinge-se um desses alvos essa variável passava a ter acrescentado o valor 1. Logo a condição de vitória para este nível passou a ser o projétil estar nas coordenadas do último alvo e ambas as variáveis S1 e S2 terem o valor igual ou superior a 1.

3.5. Nível 4



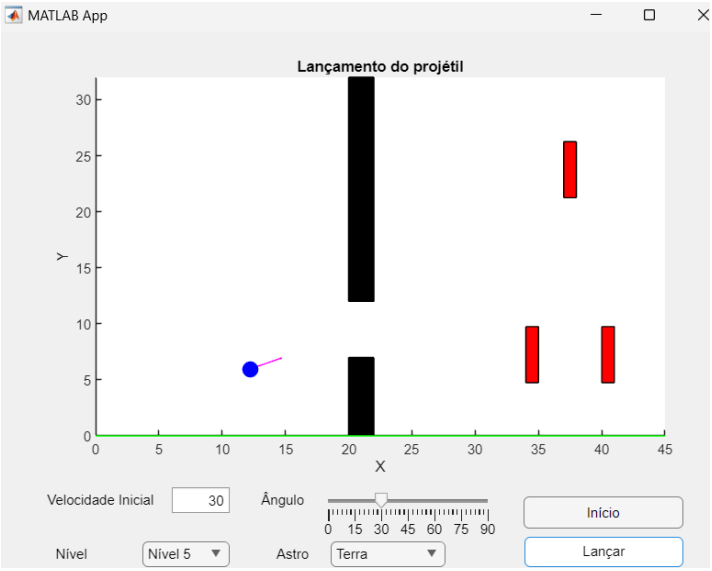


No nível 4, o nosso objetivo era fazer um alvo que se desloque verticalmente com o intuito de aumentar a dificuldade do jogo.

Ao implementar esta ideia deparámo-nos logo com um problema, para o nosso plano resultar, o programa deveria guardar a informação da posição atual do alvo no momento em que o jogador seleciona a opção 'Lançar'. Para resolver este problema, adicionámos uma *property* na qual criámos variáveis globais, isto é, uma variável que quando é atualizada num *callback,* atualiza automaticamente em todos os demais, desta forma, conseguimos transferir a informação desejada.

Para fazer a animação do movimento do alvo, definimos a sua velocidade e criámos uma expressão *if-elseif* para garantir que o alvo oscilasse entre as posições desejadas, o alvo iniciava o seu movimento na sua posição inicial e deslocava-se no sentido positivo do y, quando este alcançasse determinada posição, o sentido da velocidade trocava e este passava a deslocar-se no sentido negativo do y, até chegar a determinada posição, onde a velocidade voltava a trocar de sentido. Isto repetia-se infinitamente até o jogador ganhar ou perder o jogo.

3.6. Nível 5



Para finalizar, no nível 5 decidimos juntar todas as nossas ideias anteriores em único nível de dificuldade extrema, combinando a parede com os múltiplos alvos móveis obrigatórios de acertar.

4. Conclusão

Concluindo, podemos dizer que o nosso objetivo de simular e modelar o lançamento de um projétil foi um sucesso. Foi possível criar uma app capaz de simular lançamentos conforme a realidade e as suas leis físicas. Apesar de que alguns fatores como a resistência do ar, e outras condições atmosféricas tornariam esta simulação ainda mais próxima da realidade, o que seria um possível desenvolvimento futuro.

Uma implicação que os resultados dos lançamentos confirmou, é o facto de que tendo conhecimento das leis físicas e as suas equações correspondentes ao movimento, é possível prever com certeza qual será o local onde um projétil lançado com X velocidade inicial e Y ângulo inicial irá cair. E este conhecimento acaba por ser útil para concluir os níveis disponibilizados na app.

A app final acabou por ser composta por 6 níveis diferentes, um deles sendo apenas um simulador não contendo um objetivo de vitória. Outro possível desenvolvimento futuro seria transformar esta app num jogo completo, contendo mais níveis, recompensas ao vencer, entre outras coisas comuns de videojogos.